CMPS 3390 Project 2

Team Members: Jaquelin Xiomara Burgos Hernandez, Ibran Perez, Kyle John Carl Duncan

1. Project Overview

For our project we decided to make an app that lets users search for video games, view information about them, and organize the games into named lists. The app is built with Flutter. A Node.js and Express server connects the app to a PostgreSQL database hosted on Supabase. Game Information, such as covers, descriptions, ratings, and release dates, all come from the RAWG API. Users create an account and log in. Passwords are hashed with Argon2id, and the server issues a JSON Web Token (JWT) that the app sends with every request, so each user can only access their own lists.

2. Part 1: Research and Information Gathering

2.1 Team Communication and Work Process

- Team Communication: Discord
- Check-ins: Every Tuesday after the lab ends (12:30 PM or later)
- Task Management: Tasks are divided among team members based on the parts of the project they are responsible for. We communicate with each other to discuss progress, ask questions, and coordinate any changes.
- Responsibilities: Ibran worked on the database, server, and search screen; Jaqui worked on UI for lists view and log in view, app navigation ; Kyle worked on user authentication and authorization on server and connecting auth to the app
- The team uses Git and a public GitHub repository. Changes are developed in branches and merged into the main branch after review. Secrets such as API keys and database connection details are kept out of the repository.
- Project documentation is being created in Google Docs and will be uploaded to the GitHub repository as a PDF once it is complete.

2.2 What Success Looks like

The project is successful when users can log in, search for games, view game details, create and manage lists, add or remove games, and keep their data between sessions. The app should have a simple, user-friendly GUI that is easy to navigate. Inputs should be validated, and the app should display clear messages when something goes wrong.

2.3 Target User

The target user is a casual gamer who discovers games through friends or videos and wants a simple way to keep track of games they want to try. They may also want to create personal lists of games for fun.

- Organize games into personal lists, such as "Backlog" or "Co-op Games."
- Search for games and view their information without leaving the app.
- Sign in so their game lists are saved and associated with their account.

2.4 Main User Scenarios

1. Create an account using the required information.
2. Log in using an existing account to access saved game lists.
3. Search for a game by title and view matching results, including its cover image, name, and release date.
4. Select a game to view its available details.
5. Create a named list and view it along with its game count.
6. Add a game to a list and receive a message confirming whether it was added or was already there.
7. Open a list to view its games or remove a game.
8. Rename or delete a list. The app asks for confirmation before deleting a list and its entries.
9. Log out, which returns the user to the Log In screen. The logged-out session can no longer be used.
10. Close and reopen the app while staying logged in.

2.5 Main Features and Priorities

- Game search: Search for games using RAWG, including loading indicators and messages for empty results or errors.
- Game details: Display available game information from RAWG in a popup.
- Game lists: Create, View, rename (Update), and Delete lists.
- List entries: Add games to lists, view games within a list, and remove games.
- User interface: Provide a home screen for searching games and a list screen for viewing and managing game lists.
- Persistent storage: Store data using Supabase PostgreSQL so that user lists remain available between sessions.
- User accounts: Users sign up and log in with a username and password. Passwords are hashed, each request carries a JWT, and users can only access their own lists. Users stay logged in between app launches until they log out.
- Input validation: Validate user input in the app, server, and database.

The team focused on core features first. Changing a game's status (Want to Play, Playing, or Finished) and a “Recently Created” feed for discovery and user interaction, showing which lists already contain a game, social features, personal notes, and additional design improvements were deferred to keep the project within scope.

2.6 Requirements in Plain Language

- Users can create an account, log in, and log out.
- Each user can only see and change their own lists.
- Users can search for games by name and view the matching results.
- Users can select a game to view its available details.
- Users can create, rename, and delete their own lists.
- Users can add games to lists and remove them later.
- The app displays the number of games in each list.
- User data is saved and remains available between sessions.
- The app validates inputs and displays clear error messages.

3. Part 2: Environment Setup

3.1 Technology and Tools

- IDE: Visual Studio Code
- Framework: Flutter
- Server: Node.js with Express
- Database: PostgreSQL hosted on Supabase
- Authentication: argon2 (password hashing) and jsonwebtoken on the server; flutter_secure_storage (saved login) in the app
- Game information: RAWG API
- Database access: the server uses the pg package and dotenv
- Version Control: Git and GitHub
- Database diagram: dbdiagram.io
- Team Communication: Discord

3.2 Development Environment

- Team members develop the application on Mac laptops.
- The database is hosted on Supabase, allowing team members to connect to a shared database instead of maintaining separate local databases.
- The database schema is stored in server/db/cmps3390DBSchema.sql.
- Each developer creates their own lib/config.dart file for the RAWG API key and server/.env file for database connection settings, using the example files provided in the repository. The server/.env file holds JWT_SECRET, a random 256-bit key generated with npm run setup.

3.3 Repository and Team Workflow

The repository contains the Flutter app in lib, the backend in server, and documentation in docs. Team members make changes in small branches, communicate progress in the group chat, and merge reviewed changes into main. API keys, database credentials, and generated dependencies are excluded from the version control.

4. Part 3: Application Development:

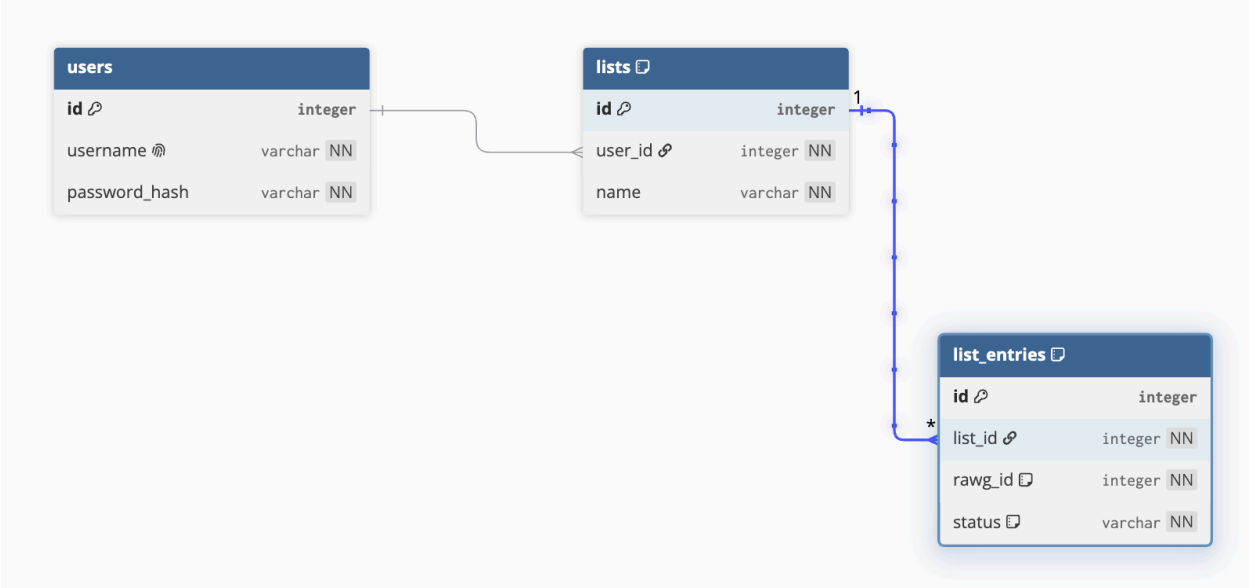
4.1 Application Structure

The app uses a Model-View-Controller (MVC) structure. Views display the interface, controllers handle user actions, and models manage data operations. The Flutter app communicates with the Node.js/Express server for list operations. The server validates requests and performs SQL queries against Supabase PostgreSQL. The app also calls RAWG for game searches and game details.
The app includes Home, Search, and Lists tabs, plus popups for game details, viewing a list's games, adding a list, and editing a list. Login and sign-up screens are also part of the planned interface.

4.2 Database Design

One user can have multiple lists, and each list can contain multiple game entries. Deleting a user deletes their lists, and deleting a list deletes its entries. The database stores the RAWG game ID rather than copying the game's full information; the app retrieves game details from RAWG.

Note: The status field is included in the database design and diagram but was not implemented as a user-facing feature in the application. Support for tracking game status may be added in a future update.



4.3 Input Validation and Data Protection

- User input is validated to prevent empty or invalid list names and search queries.
- SQL queries use parameterized values to help prevent SQL injection.
- The server checks list ownership to ensure users can only access their own lists.
- Database constraints prevent duplicate usernames, list names, and games within the same list.
- Errors are handled with clear messages without exposing sensitive server details.

4.4 Important Technical Decisions

- The server handles database queries, validates inputs, and checks list ownership.
- Only RAWG game IDs are stored in the database to avoid duplicating game information.
- The RAWG API key is stored in a Git-ignored file. However, it may still be extracted from the built app. Moving API requests to the server could improve security.
- Game details are cached during a session to reduce repeated API requests.
- Deleting a list requires confirmation because its game entries are also deleted.

4.5 Server Routes

The server provides routes to check the database connection and manage users' game lists.

- GET /health — Check the database connection.
- GET /users/:userId/lists — Retrieve a user's lists and game counts.
- POST /users/:userId/lists — Create a list.
- PATCH /users/:userId/lists/:listId — Rename a list.
- DELETE /users/:userId/lists/:listId — Delete a list.
- GET /users/:userId/lists/:listId/games — Retrieve games in a list.
- POST /users/:userId/lists/:listId/games — Add a game to a list.
- DELETE /users/:userId/lists/:listId/games/:rawgId — Remove a game from a list.